

Organização e Arquitetura de Computadores

Jorge Augusto Salgado Salhani

no. USP: 8927418

Maio, 2026

1 Lista 3

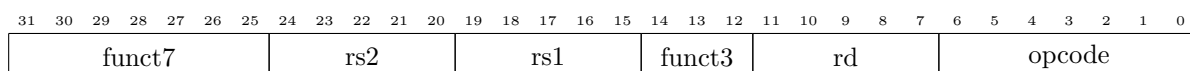
1. Quais são os tipos de instruções da arquitetura RISC-V, no conjunto de instruções RV32I?

[SOLUÇÃO]

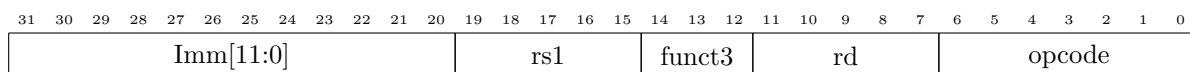
A ISA RV32I (32-Integer Base Instruction Set Architecture) apresenta 4 formatos de instruções principais (R/I/S/U), com 2 variantes baseadas em valores imediatos (B/J), todas com tamanho de 32 bits. São os tipos

Principais:

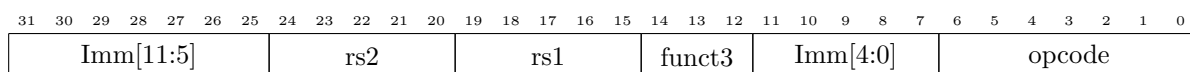
- **R (Register): add rd, rs1, rs2**



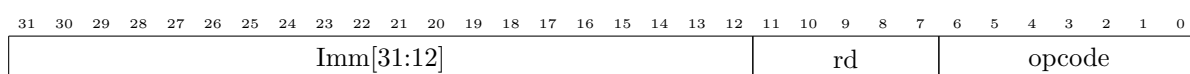
- **I (Immediate): addi rd, rs1, imm**



- **S (store): sw rs2, offset(rs1)**

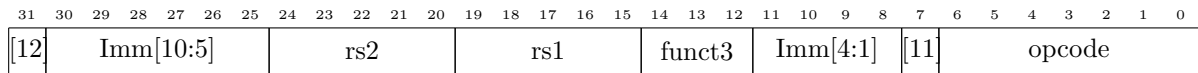


- **U (Upper Immediate): lui rd, imm**

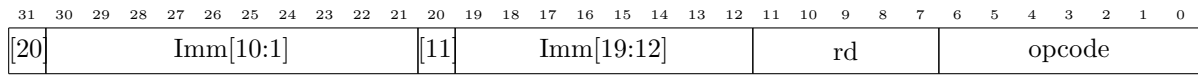


Variantes de imediatos:

- **B (Branch): beq s1, s0, label**



- **J (Jump): jal, s0, label**



2. Decodifique as seguintes instruções de máquina do RISC-V: 0x00500893, 0x000e8533, 0x00afa023 (em hexadecimal) e determine o que elas fazem.

[SOLUÇÃO]

Conversão de hexadecimal 0x00500893 para binário:

0	0	5	0	0	8	9	3
0000	0000	0101	0000	0000	1000	1001	0011

Temos a sequência 0000 0000 0101 0000 0000 1000 1001 0011. O opcode de 7 bits é dado por opcode: 0010011, sendo portanto do tipo I.

opcode	tipo
0110011	R
0010011	I
0000011	I
0100011	S
1100011	B
1101111	J
1100111	I
1110011	I

Utilizando do modelo tipo I, temos Assim

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Imm[11:0]												rs1				funct3			rd			opcode									
0000 0000 0101												0000 0				000			1000 1			001 0011									

$$rd : 17 \ (x17 = a7)$$

$$funct3 : 0$$

$$rs1 : 0 \ (x0 = zero)$$

$$Imm[11 : 0] : 5$$

Com base na tabela de referência, o par (opcode, funct3) indica a função **addi**. Logo, temos

$$0x00500893 \rightarrow \text{addi a7, zero, 5}$$

Para a sequência 0x000e8533, temos em binário

$$\left\| \begin{array}{c|c|c|c|c|c|c|c} 0 & 0 & 0 & e & 8 & 5 & 3 & 3 \\ \hline 0000 & 0000 & 0000 & 1110 & 1000 & 0101 & 0011 & 0011 \end{array} \right\|$$

O opcode de 7 bits é dado por 0110011, sendo portanto do tipo R.

Utilizando o modelo tipo R, temos

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
funct7							rs2					rs1					funct3			rd				opcode							
0000 000							0 0000					1110 1					000			0101 0				011 0011							
0							0					x29					x0			x10				-							

O par (opcode, funct3) = (R, 0x0) codifica a instrução **add**. Como $rd = x10 \rightarrow a0$, $rs1 = x29 \rightarrow t4$ e $rs2 = x0 \rightarrow zero$, temos

$$0x000e8533 \rightarrow \text{add a0, t4, zero}$$

Para a sequência 0x00afa023, temos

$$\left\| \begin{array}{c|c|c|c|c|c|c|c} 0 & 0 & a & f & a & 0 & 2 & 3 \\ \hline 0000 & 0000 & 1010 & 1111 & 1010 & 0000 & 0010 & 0011 \end{array} \right\|$$

Sendo opcode os 7 bits menos significativos, 0100011. A instrução segue o modelo S.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Imm[11:5]							rs2					rs1					funct3			Imm[4:0]				opcode							
0000 000							0 1010					1111 1					010			0000 0				010 0011							
0000 000							x10					x31					x2			0000 0				x23							

O par (opcode, funct3) = (S, x2) codifica a instrução **sw**. Como $rs2 = x10 \rightarrow a0$, $rs1 = x31 \rightarrow t6$ temos

$$0x00afa023 \rightarrow \text{sw t6, 0(a0)}$$

3. Converta de Assembly RISC-V para Binário ou de Binário para Assembly RISC-V, conforme o caso, as instruções solicitadas abaixo. Expresse as instruções binárias em hexadecimal, quando for o caso, na coluna mais à direita. Expresse as instruções em Assembly, quando for o caso, na coluna "Instrução em Assembly". A primeira coluna indica o endereço de memória (em hexadecimal) que a instrução se encontra.

End. Byte na RAM	Instr. Assembly	Instr. Binário	Instr. hexadecimal
8			0x00844433
20	beq s0, zero, nao		
36	nao: addi a0, zero, 10		
44			0x00052403

Primeiro, convertendo em binário as instruções em hexadecimal.

Caso 0x00844433

0	0	8	4	4	4	3	3
0000	0000	1000	0100	0100	0100	0011	0011

Com opcode de 7 LSB, temos opcode = 0110011, relativo à instrução do tipo R.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
funct7							rs2					rs1					funct3			rd					opcode						
0000 000							0 1000					0100 0					100			0100 0					0110011						
x0							x8					x8					x4			x8					x51						

Com o par (opcode, funct3), temos (R, x4) = XOR. Em relação aos registros, temos x8 = s0. Assim, nossa instrução é

0x00844433 → xor s0, s0, s0

Caso 0x00052403

0	0	0	5	2	4	0	3
0000	0000	0000	0101	0010	0100	0000	0011

Com opcode dos 7 LSB, temos opcode = 0000011, relativo à instrução de tipo I

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Imm[11:0]												rs1				funct3			rd				opcode								
0000 0000 0000												0101 0				010			0100 0				000 0011								
x0												x10				x2			x8				x3								

Com o par (opcode, funct3) = (x3, x2) temos a instrução lw. Para os registradores, temos x10 = a0, x8 = s0. Assim, temos

0x00052403 → lw s0, a0, zero

Agora, para a instrução beq s0, zero, nao.

Temos tipo B, com opcode 1100011. A operação beq apresenta funct3 = x0 = 000.

Para os registradores, temos s0 = x8, zero = x0. O valor imediato é indicado pelo label **nao**, cujo endereço de byte na RAM é 36. Logo Imm = x36

Convertendo x36 em binário, temos x36 = 100100 = 0000 0010 0100. Sendo os bits considerados de 1 a 12, desconsideramos o LSB e temos x36 = 100100 = 0000 0001 0010(0)

A partir do template para a tipo, B, temos

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
[12]	Imm[10:5]					rs2					rs1					funct3					Imm[4:1]					[11]	opcode				
[12]	Imm[10:5]					x0					x8					000					Imm[4:1]					[11]	1100011				
0	000001					00000					01000					000					0010					0	1100011				

Pois como Imm é quebrado, temos para x36 = 0000 0010 0100 que:

Imm[12]	0
Imm[11]	0
Imm[10:5]	000001
Imm[4:1]	0010

Logo, temos a conversão

beq s0, zero, nao → 0000 0010 0000 0100 0000 0010 0110 0011 → 0x02040263

Para a última instrução, temos addi s0, zero, 10.

Para addi, temos opcode = 0010011, com funct3 = 000. Para os registradores, temos rs1 = x0 e rd = s0 = x8. Para o valor imediato, temos x10.

Utilizando o template para instrução do tipo I, temos Logo, temos a conversão

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Imm[11:0]												rs1				funct3				rd				opcode							
x10												x0				000				x8				0010011							
000000001010												00000				000				01000				0010011							

addi s0, zero, 10 → 0000 0000 1010 0000 0000 0100 0001 0011 → 0x00a00413

Por fim nossa tabela fica

End. Bytena RAM	Instr. Assembly	Instr. Binário	Instr. hexadecimal
8	xor s0, s0, s0	00000000100001000100010000110011	0x00844433
20	beq s0, zero, nao	0000001000000010000000001001100011	0x02040263
36	nao: addi a0, zero, 10	000000001010000000000010000010011	0x00a00413
44	lw s0, a0, zero	000000000000001010010010000000011	0x00052403